

Project Executable Files

Date	5 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

Project Structure

Rising Water/
├── app.py                ← Flask web application (main server)
├── train_model.py        ← ML pipeline (EDA, preprocessing, model training)
├── deploy.py             ← Automated PythonAnywhere deployment script
├── locustfile.py         ← Locust performance testing script
├── requirements.txt      ← Python dependencies
├── floods.save           ← Serialized XGBoost model (Joblib)
├── scaler.save           ← Serialized StandardScaler (Joblib)
├── transform.save        ← Serialized StandardScaler copy (Joblib)
├── database.db           ← SQLite database (users + history)
├── dataset/
│   ├── rainfall_india.csv ← Training dataset (115 rows x 11 columns)
│   └── create_dataset.py  ← Dataset generator script
├── model/
│   ├── floods.save       ← Model backup copy
│   └── scaler.save       ← Scaler backup copy
├── static/
│   ├── cstc/
│   │   └── style.css     ← Application stylesheet
│   ├── images/          ← Generated EDA & model charts (PNGs)
│   └── js/               ← JavaScript directory
└── templates/
    ├── home.html         ← Landing page
    ├── index.html        ← Prediction input form
    ├── chance.html       ← Flood predicted result
    ├── no_chance.html    ← No flood result
    ├── analysis.html     ← EDA charts dashboard
    ├── history.html      ← Prediction history logs
    ├── login.html        ← Login page
    ├── register.html     ← Registration page
    ├── predict.html      ← Alternate predict page
    ├── flood.html        ← Flood info page
    ├── no_flood.html     ← No flood info page
    ├── preprocessing.html ← Preprocessing details page
    └── model_building.html ← Model building details page
  
```

Step 3: Deployment / Access Details :

Hosted / Deployed URL	Hosted Link : https://khushwanth.pythonanywhere.com/
Login Credentials (Demo)	Can Sign in with your credentials if registered ; If not registered then Sign up (Public Application)
Platform / Hosting Provider	Python Anywhere
Repository Link	Project Repo Link : https://github.com/AP24110010739/Rising-Waters
Demo Video Link	https://drive.google.com/drive/folders/1CGLgoU0ei-Rb_Hp6GTidY0a9IP6OKPX0?usp=sharing

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- Python 3.10 or later
- pip (Python Package Manager)
- Git (optional)
- Internet connection
- Visual Studio Code / PyCharm (recommended)

Step 1: Clone the repository

```
git clone <Repository_Link>  
cd Rising-Water
```

Step 2: Install the required dependencies

```
pip install -r requirements.txt
```

Step 3: Verify the project files Ensure the following files are available:

- app.py
- train_model.py
- floods.save
- scaler.save
- transform.save
- requirements.txt
- templates/
- static/
- dataset/rainfall_india.csv

Step 4: Train the ML model (first time only)

```
python train_model.py
```

Step 5: Run the Flask application

```
python app.py
```

Step 6: Open the application Open the following URL in your web browser: <http://127.0.0.1:5001>

Step 7: Register and Login

- Click on Register and create a new account with username, email, and password.
- Login using the registered credentials.

Step 8: Test the application

- Navigate to the Predict page.

- Enter the required weather values (Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec).
- Click Predict Flood.
- View the prediction result (Flood Chance / No Flood Chance) with the probability percentage.

Step 9: View Analysis and History

- Navigate to Analysis to view EDA charts, heatmaps, and model comparisons.
- Navigate to History to view all past prediction logs with timestamps and user details.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on quality and size of the dataset	Can be improved by training on larger, real-world meteorological datasets from sources like IMD or OpenWeather.
2	Works only with manually entered weather parameters.	Future enhancement can integrate live weather APIs (such as OpenWeatherMap) to auto-fetch real-time data.
3	No password reset and verification functionality	Can be added using Flask-Mail with OTP-based verification in future versions.